EJERCICIO PRÁCTICO Nº 3.2

Objetivos

- Comprender el funcionamiento de los Sockets como interfaz de comunicación entre aplicaciones en red, identificando el rol de las direcciones IP, los puertos y el protocolo de transporte.
- Implementar una aplicación cliente-servidor con sockets en un lenguaje de programación a elección, con capacidades de paso de mensajes entre varios clientes.
- Verificar la interoperabilidad entre aplicaciones desarrolladas en distintos lenguajes mediante un protocolo de mensajes común.

Ejercicio 1:

Se requiere realizar la programación e implementación de un chat básico mediante el uso de sockets. Para el desarrollo del mismo se puede codificar en lenguaje de programación a elección: Java, Python, C/C++, C#, etc.

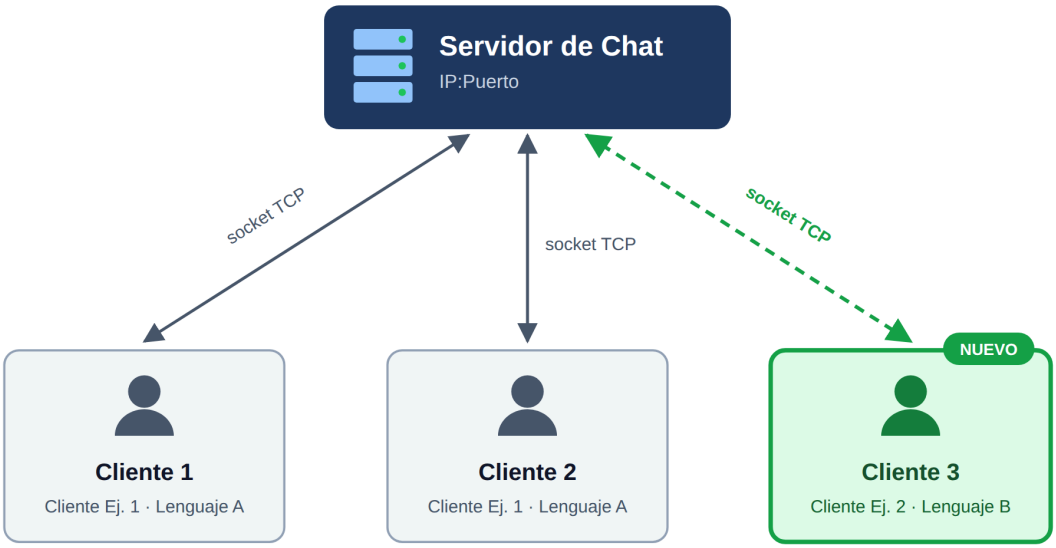
Actividades:

1. Diseñar y desarrollar una aplicación de chat en red utilizando sockets para permitir la comunicación en tiempo real entre varios usuarios.
2. Crear una aplicación de chat en la que al menos dos usuarios puedan comunicarse a través de una red local o a través de Internet utilizando sockets.
3. Implementar una interfaz de usuario que permita a los usuarios ingresar sus nombres de usuario y mensajes, así como ver los mensajes recibidos de otros usuarios en tiempo real.
4. Utilizar sockets para establecer una conexión entre los clientes y un servidor central. El servidor debe ser capaz de recibir mensajes de los clientes y retransmitirlos a todos los demás clientes conectados.
5. Manejar adecuadamente los errores de conexión y desconexión de clientes, y proporcionar una mejora en la experiencia de chat.
6. Implementar comandos especiales en el cliente, como "*/listar*" para ver la lista de usuarios conectados al chat y "*/quitar*" para desconectarse del servidor.
7. Se debe adjuntar un informe explicativo en donde se indiquen de manera detallada las líneas de código donde se establece la comunicación entre el cliente y el servidor a través del uso de sockets (Librerías importadas, puertos utilizados, etc.).
8. Se debe adjuntar a la entrega, junto con el informe explicativo el código fuente del cliente y el servidor. Se deben incluir capturas de pantalla del chat en ejecución, interacción entre diferentes clientes.

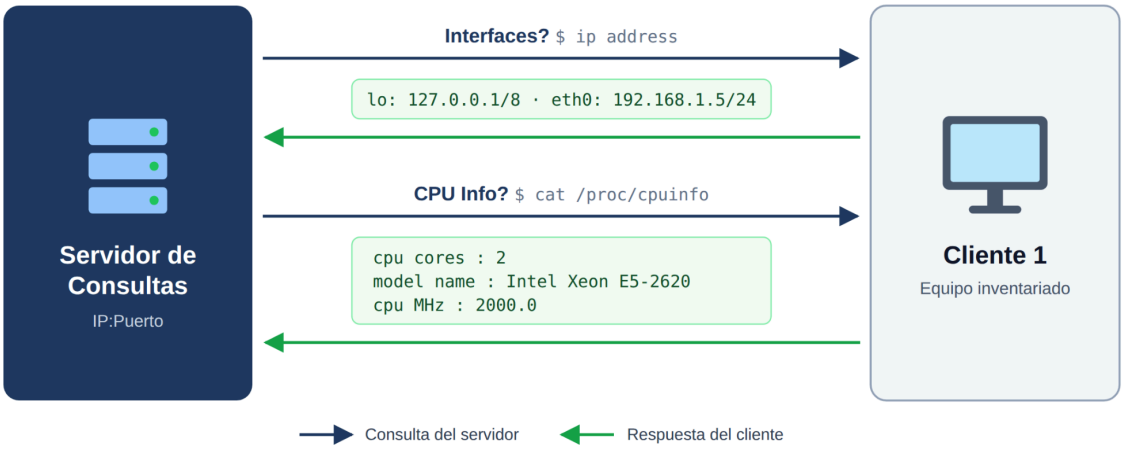
Ejercicio 2:

Utilizando como base el desarrollo del ejercicio N°1: diseñar e implementar un programa, basado en la comunicación por sockets, sobre alguno de los siguientes escenarios:

1. **Nuevo Cliente de Chat**
   Se requiere la programación de un nuevo cliente de chat, en un lenguaje diferente al seleccionado en el ejercicio 1, para este punto el mismo debe poder conectar al servidor desarrollado en el punto anterior.
   El nuevo cliente de chat debe solicitar al usuario el ingreso de parámetros: Host IP y puerto del servidor, luego mostrar con un mensaje si la conexión ha sido exitosa.
   El comportamiento y funcionalidades deben ser las mismas que el cliente desarrollado en el ejercicio 1.



2. **Servidor de consultas de características de HW/SW para inventario:**
   Servidor socket que realiza diversas consultas sobre las características del cliente como ser:
   - características del procesador.
   - memoria RAM.
   - estado del disco.
   - filesystem.
   - promedio de carga (loadavg).
   - particiones, versión del sistema operativo.
   - interfaces de red, procesos ejecutándose.



Ver: directorio /proc con muchos datos, comandos: 'hostnamectl', 'hostname', 'ip address', 'free', 'ps', etc.

Ejercicio 3:
Inspección en Wireshark

Mediante la herramienta Wireshark, analizar el flujo de los paquetes cuando se establecen las conexiones de varios clientes con el servidor.

- Identificar los puertos intervinientes y/o direcciones IP. Indicar las tuplas de IP y puerto de origen y destino de la conexión.
- Visualizar el flujo de datos no cifrado. Enviar un mensaje con un texto identificable como la LU y capturarlo con la herramienta.

Entrega: El alumno debe entregar un documento que indique las configuraciones y verificaciones realizadas. Nomenclatura del documento de texto a entregar: "TP3.2_LUApellidoNombre".

Rúbricas evaluativas

Criterio	Excelente (100%)	Bueno (75%)	A mejorar (50%)	Insuficiente (0-25%)
1. Comunicación cliente-servidor mediante sockets	Servidor central operativo que acepta múltiples clientes en forma concurrente (hilos, select o asincronía) y retransmite los mensajes a todos los demás clientes conectados. Probado con 3 o más clientes, en red local o Internet.	Servidor y clientes se comunican correctamente y se retransmiten los mensajes, con errores menores (ej. el remitente recibe su propio mensaje, demoras o pruebas con solo 2 clientes).	Comunicación parcial: funciona solo con un cliente a la vez, no retransmite a todos o solo funciona en localhost.	No se establece la conexión por sockets entre cliente y servidor, o el código no ejecuta correctamente.
2. Interfaz y comandos del chat	El usuario ingresa su nombre y envía/recibe mensajes en tiempo real identificando al emisor. Los comandos /listar y /quitar funcionan correctamente.	Interfaz funcional con envío y recepción en tiempo real. Uno de los comandos presenta fallas menores.	Interfaz limitada (ej. no identifica al emisor o la recepción se bloquea hasta enviar). Comandos no implementados o no funcionales.	No hay interfaz utilizable ni comandos implementados.
3. Manejo de errores y desconexiones	Se gestionan desconexiones normales y abruptas de clientes, caída del servidor, puerto ocupado y datos inválidos, sin que el servidor se detenga. Se notifica a los usuarios el ingreso y salida de participantes.	Se gestionan las desconexiones más comunes; algún caso límite genera mensajes de error no controlados, sin afectar al resto de los clientes.	Manejo de errores mínimo: la desconexión de un cliente produce excepciones o afecta el funcionamiento del servidor.	Sin manejo de errores: el servidor o los clientes finalizan ante cualquier desconexión.

4. Informe explicativo del código	Se identifican y explican las líneas donde se crean y usan los sockets (creación, bind, listen, accept, connect, envío, recepción y cierre), librerías importadas, protocolo de transporte, IP y puertos utilizados, y la estrategia de concurrencia.	Explicación correcta de las líneas principales de comunicación, con omisiones menores (ej. no justifica el puerto o no explica la concurrencia).	Explicación superficial o genérica, sin referencia a líneas concretas del código.	No se presenta informe explicativo o no corresponde al código entregado.
5. Ejercicio 2: escenario implementado	Escenario completo y funcional. Informe completo con todas las descripciones y capturas de pantallas.	Escenario funcional con omisiones menores en requisitos o en su análisis (ej. falta la comparación con UDP, o el cliente no confirma la conexión).	Escenario implementado parcialmente: funciona con fallas importantes o cubre pocos de los requisitos solicitados.	No se realizó el Ejercicio 2 o no presenta evidencia de funcionamiento.
6. Ejercicio 3: Inspección en Wireshark	Se identifica cada cliente y se completa la tabla con IP y puerto de origen y destino de cada conexión. Se complementa el informe con las capturas de pantalla.	Análisis correcto en general, con omisiones menores (ej. referencia de IP/puertos incompleta).	Capturas presentes pero con análisis incompleto: sin referencia de IP/puertos.	No se realizó la captura o no hay evidencia ni análisis del tráfico.
7. Documentación y entrega	Documento estructurado con código fuente de cliente y servidor, capturas nítidas del chat en ejecución con varios clientes interactuando y explicaciones detalladas. Archivo en formato y nomenclatura correctos.	Documento organizado en general, con leves omisiones de formato o capturas incompletas. Código y capturas presentes pero no siempre explicados.	Documento incompleto o desordenado, capturas poco claras, falta el código de alguno de los componentes o no respeta la nomenclatura.	Documento muy incompleto o sin evidencias de las actividades realizadas.